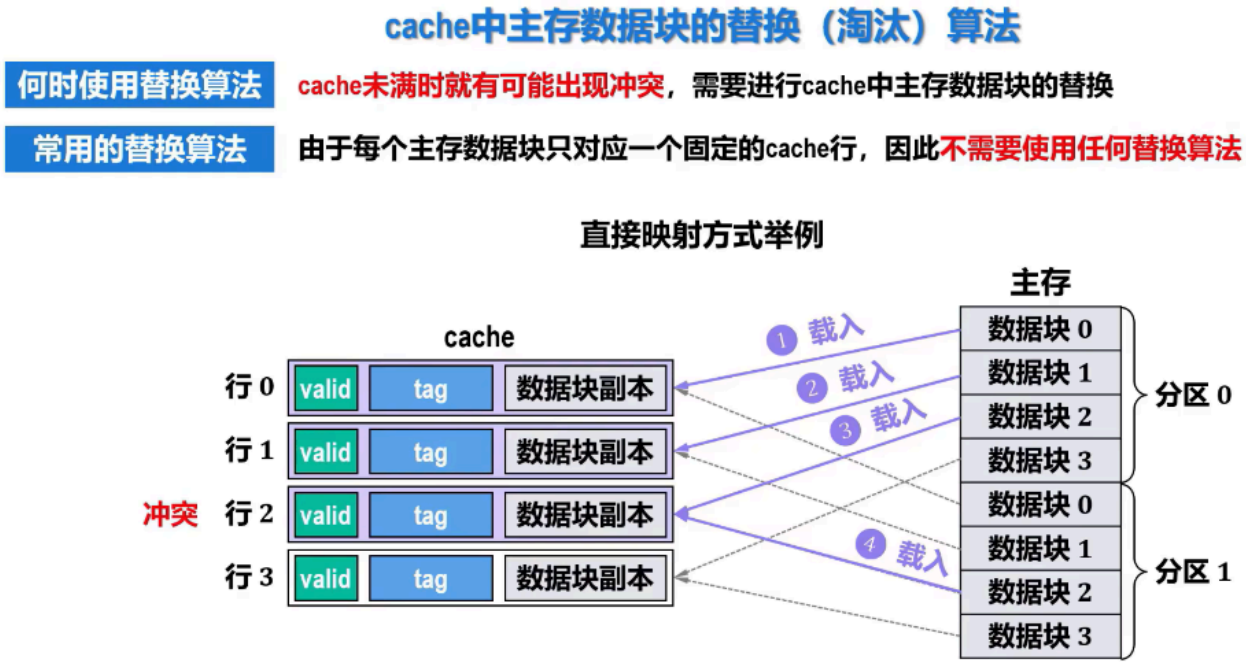
# 4.26 Cache——替换算法

**考纲定位**：计算机组成原理 · 第 4 章 存储器（4.26） | 重要度 ★★★★★ | 常见考法：选择题/大题（何时替换、FIFO/LFU/LRU、LRU 位、命中率）

**一句话核心**：直接映射无需选择；全相联满时全局选；组相联只在目标组内选；FIFO 看时间，LRU 看最近使用，LFU 看使用次数

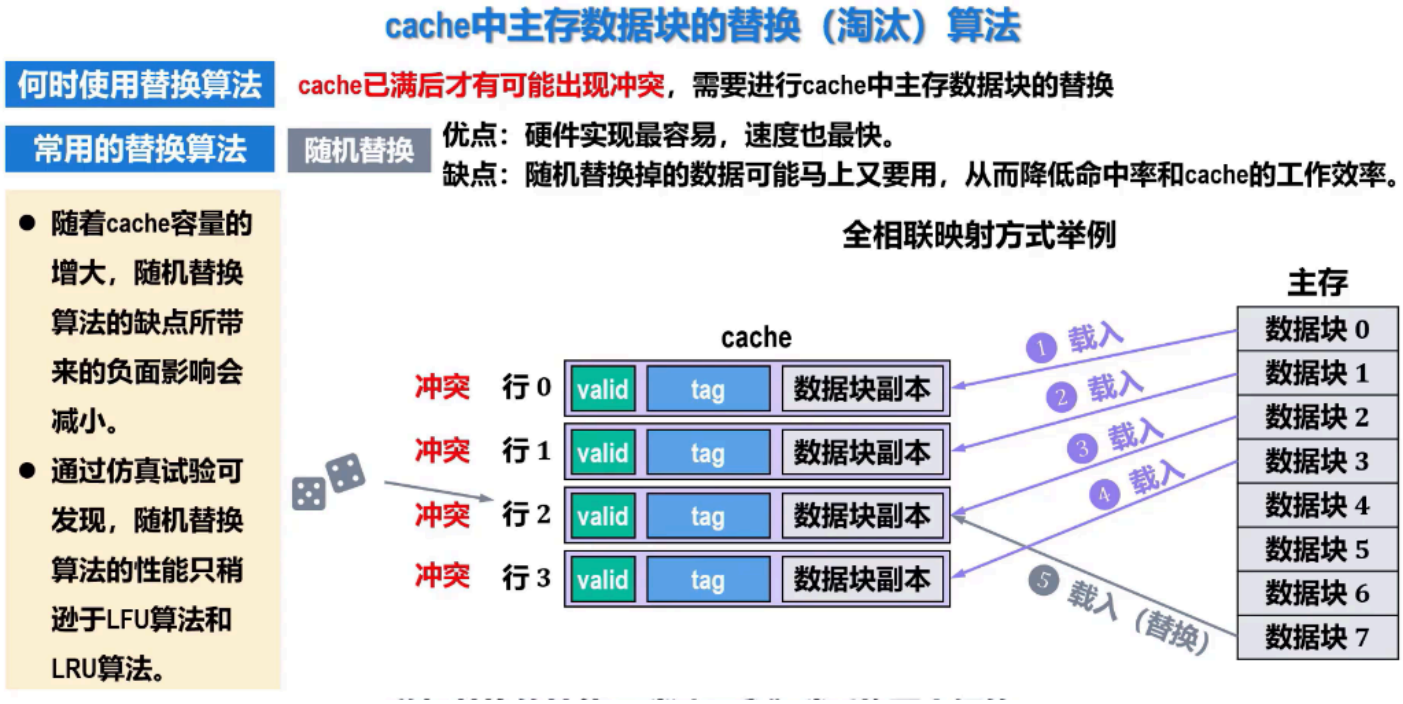
## 一、何时触发替换算法

### 1.1 直接映射



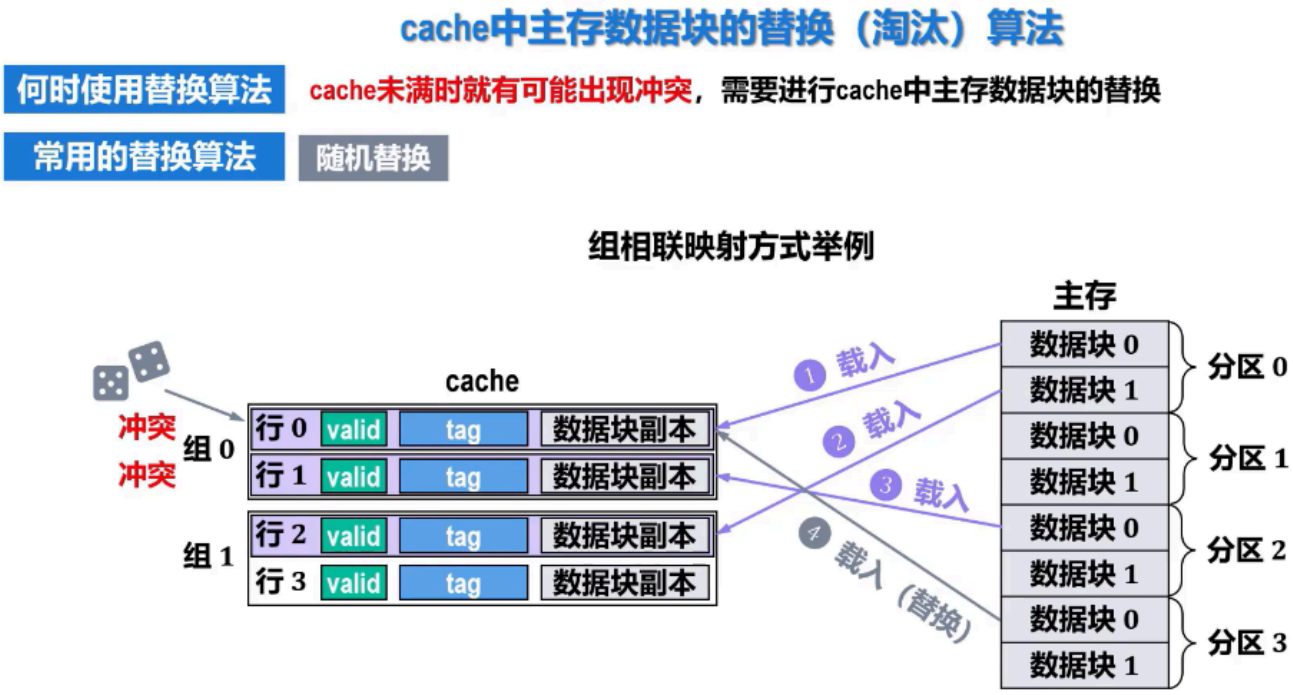

- 主存块只能映射到唯一行
- 该行被占用时，直接替换，无需选择算法
- 没有“替换哪一行”的问题，只有“是否替换”的问题

## 1.2 全相联映射




- 主存块可放到任意行
- Cache 未满时，放到空行
- Cache 满时，从所有行中选择一行替换
- 需要全局替换算法

## 1.3 组相联映射




- 主存块映射到唯一组
- 组内有空行时，放到空行
- 组满时，只在该组内选择一行替换

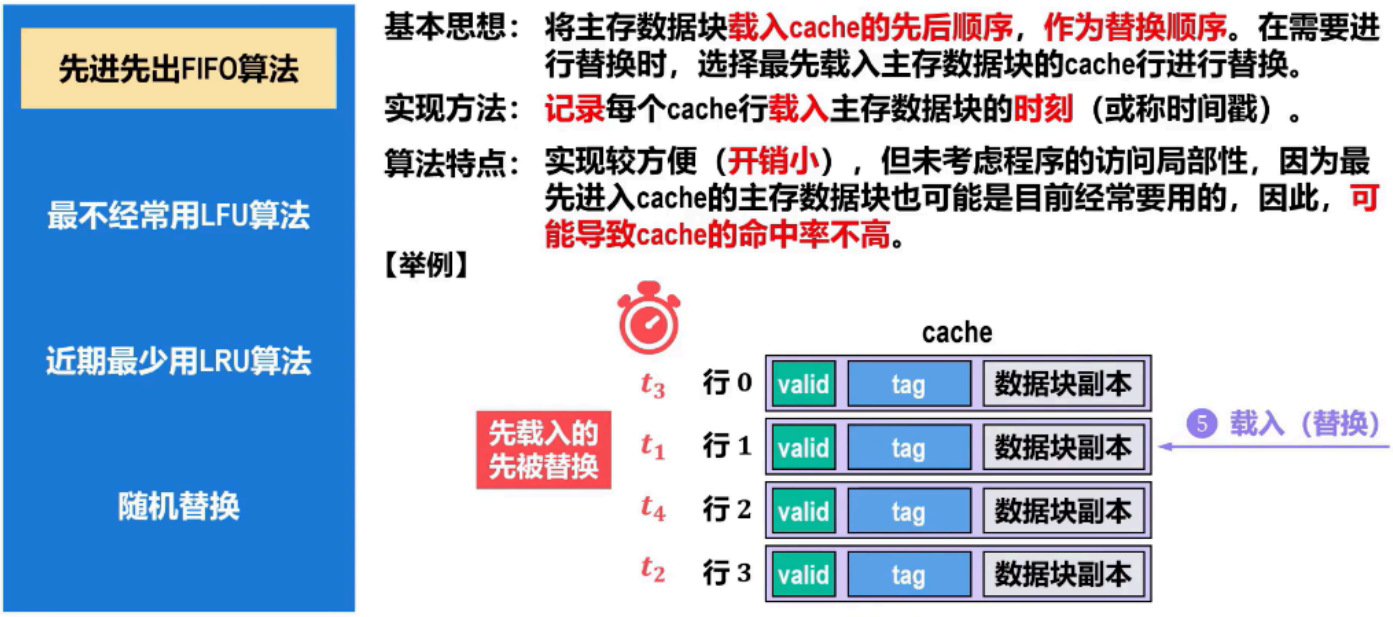
- 替换范围是组内，不是全局

**易错**

- 直接映射不需要替换算法
- 全相联和组相联都需要替换算法，但选择范围不同
- 组相联的替换只在目标组内进行

## 二、替换算法

### 2.1 先进先出 FIFO




**思想**

- 按调入 Cache 的先后顺序替换，最早调入的行先被替换
- 实现简单，开销小

**实现**

- 每行记录调入时间/时间戳，或使用循环队列
- 替换时选择时间最早的行

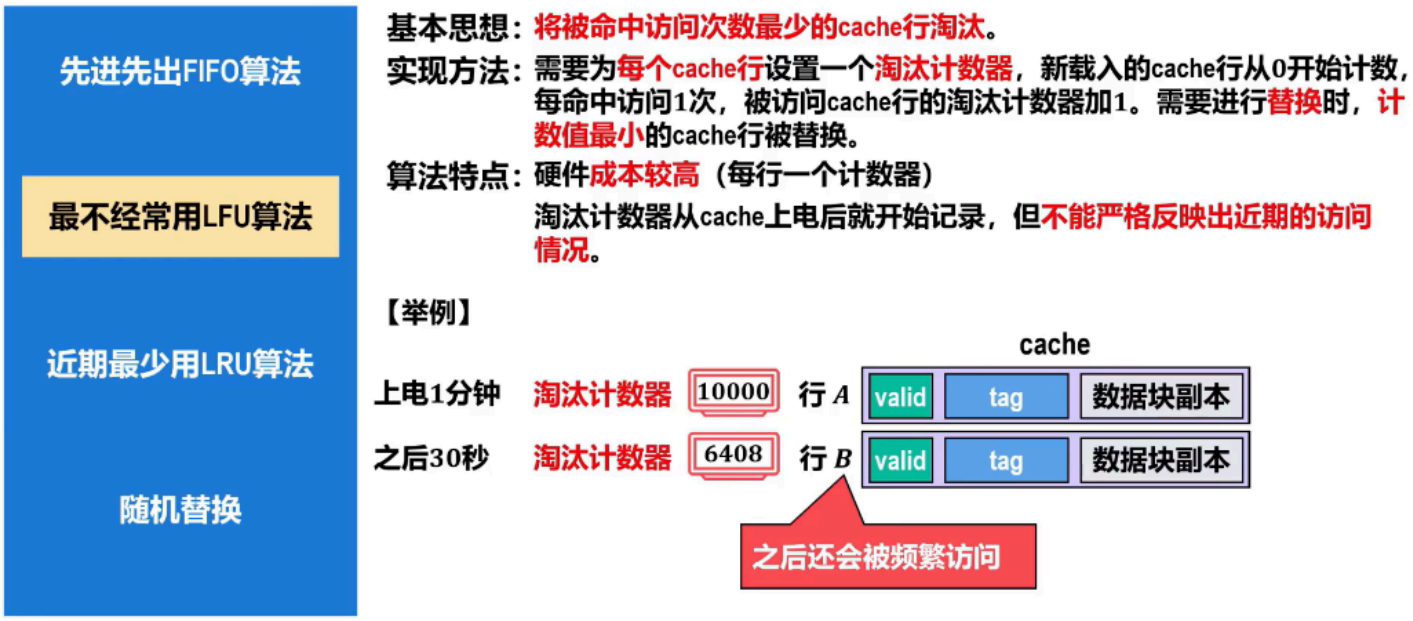
**特点**

- 没有考虑程序局部性
- 最早调入的块可能仍是热点，可能降低命中率
- 可能出现 Belady 异常（增加 Cache 容量反而命中率下降）

**易错**

- FIFO 不是“最久未使用”，而是“最早调入”
- FIFO 不考虑访问频率
- FIFO 实现简单，但命中率不一定好

## 2.2 最不经常使用 LFU




### 思想

- 记录每行被访问的次数，替换访问次数最少的行
- 考虑使用频率

### 实现

- 每行设置计数器，命中时加 1
- 替换时选择计数最小者

### 特点

- 适合访问频率差异大的程序
- 计数器需要定期清零，否则历史访问会压制新块
- 实现比 FIFO 复杂

### 易错

- LFU 看“次数”，LRU 看“最近时间”
- LFU 的计数器可能溢出，需要处理
- 频率高但已不再使用的块可能长期占用 Cache

## 2.3 最近最少使用 LRU



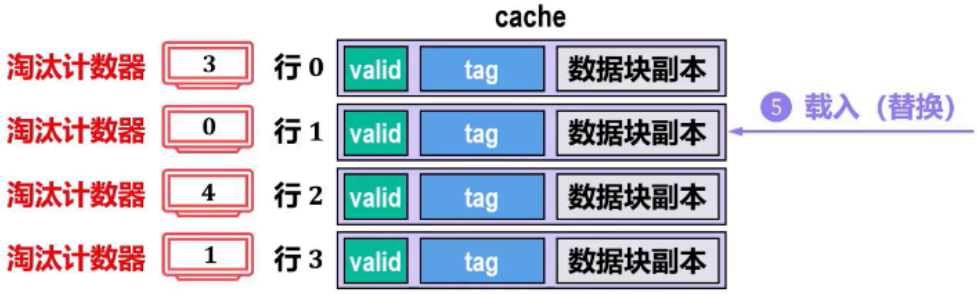

**基本思想：** 将近期内最少被命中访问过的cache行淘汰。

**实现方法：** 与LFU算法类似，也需要为每个cache行设置一个淘汰计数器。不同点是：cache行每被访问（载入/替换、命中）1次，该cache行的计数器被清零，其他计数值不为空的cache行各自的计数器被加1。需要进行替换时，计数值最大的cache行被替换。

**算法特点：** 硬件成本较高（每行一个计数器）

比较正确地反映了程序的访问局部性，因为近期最少使用的主存数据块，很可能是将来最少被访问的。

【举例】



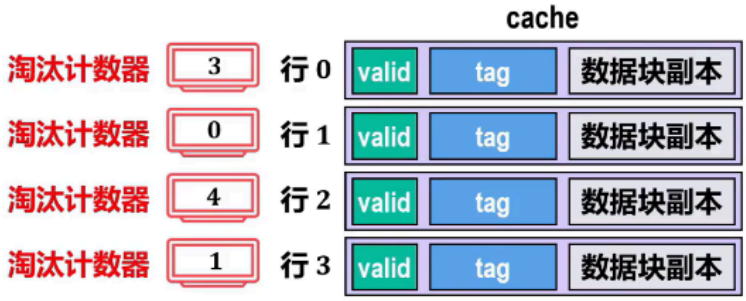

**思想**

- 替换“最长时间没有被访问”的行
- 利用时间局部性，命中率通常较好

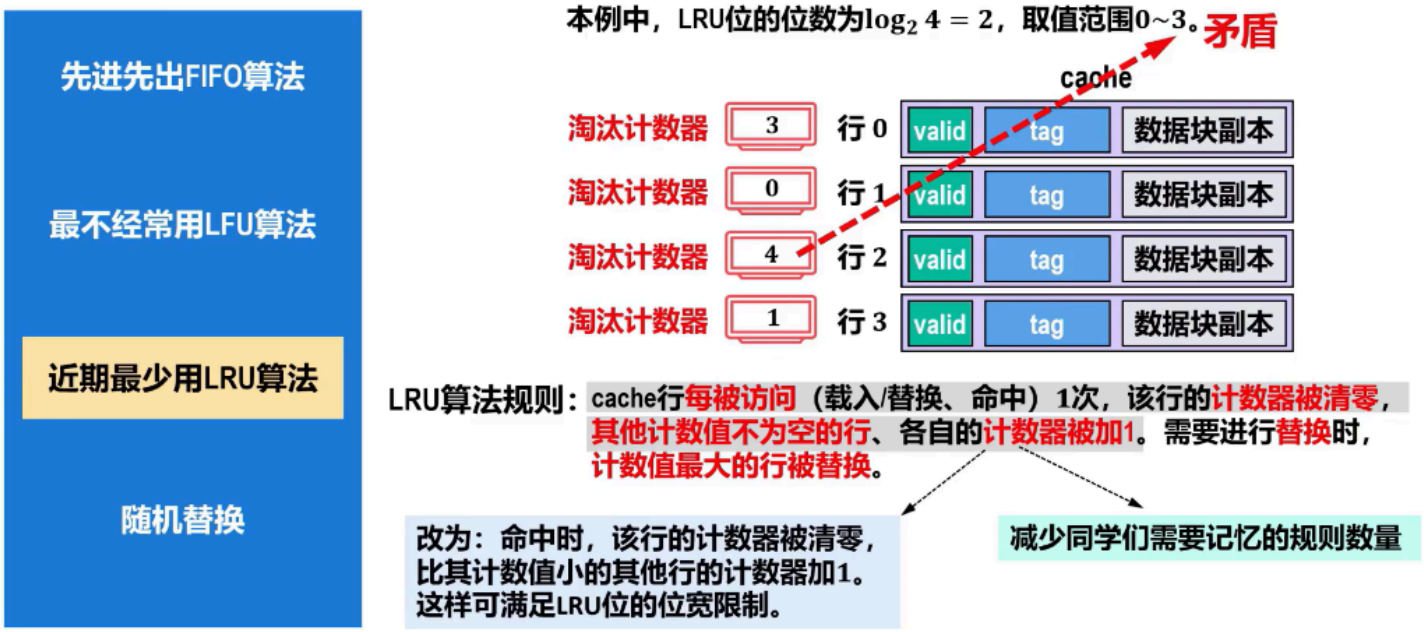
**LRU 位**

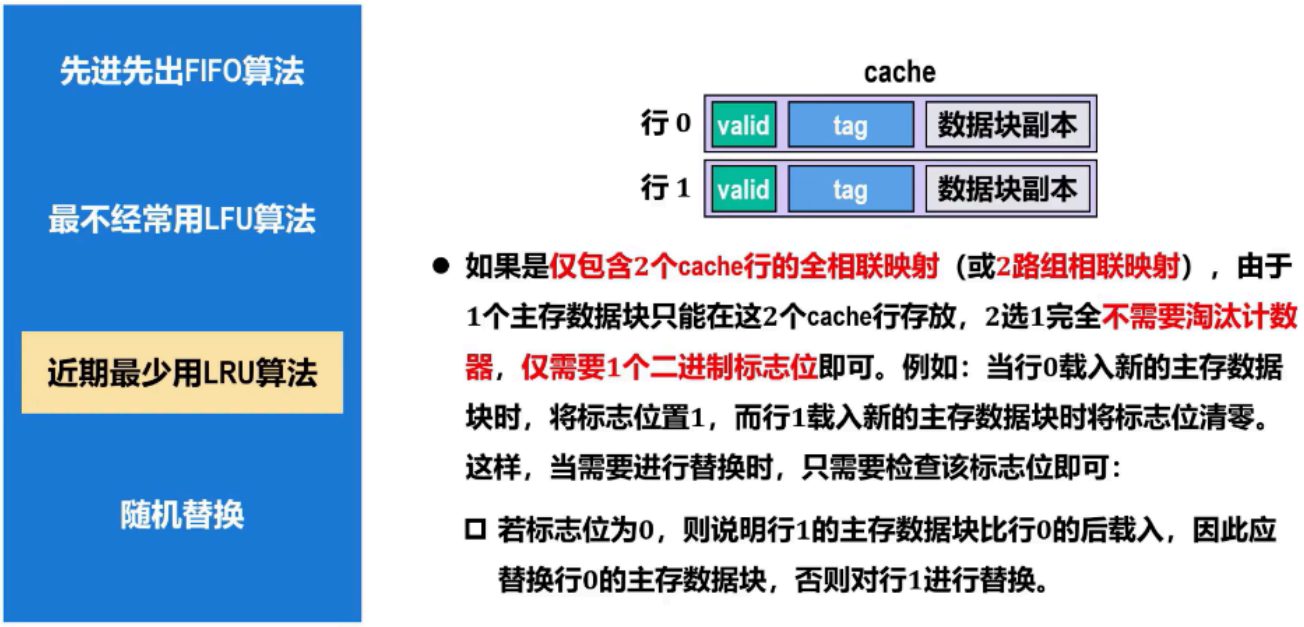



本例中，LRU位的位数为$\log_2 4 = 2$，取值范围0~3。




- 淘汰计数器的数值，称为LRU位，其位数与cache组（当只有1个组时，则为全相联映射）的大小有关。
  - 理论上，$n$路组相联（即每个cache组包含cache行的数量为$n$）时LRU位的位数为$\log_2 n$，例如，2路组相联时LRU位的位数为1，4路组相联时LRU位的位数为2。




**实现**

- 每行设置 LRU 计数器/位，记录最近使用情况
- 命中时更新该行，使其成为“最近使用”
- 替换时选择“最久未使用”的行

**LRU 位计算**

- 若每组有 $n$ 行，需要区分 $n$ 种“新旧”顺序
- LRU 位数 $= \log_2 n$（向上取整）
- 例：2 路组相联 → 1 位；4 路 → 2 位；8 路 → 3 位

**易错**

- LRU 位不是“每行一位”；它记录的是组内各行的新旧顺序
- 2 路组相联只需 1 位即可区分哪一行更久未用
- LRU 实现成本较高，实际常用近似 LRU

**三种算法对比**

| 算法 | 依据 | 优点 | 缺点 |
| --- | --- | --- | --- |
| FIFO | 调入时间 | 简单 | 不考虑局部性，可能 Belady |
| LFU | 访问次数 | 适合频率差异 | 计数器管理复杂，历史包袱 |
| LRU | 最近访问时间 | 命中率较好 | 实现成本高 |
| 随机 | 随机选择 | 最简单 | 命中率不稳定 |

**易错**

- FIFO/LFU/LRU 都不要与“写策略”混淆
- 替换算法只在全相联/组相联使用
- 直接映射的“替换”是固定的，不需要算法

**知识关联**：三种映射（4.23 Cache——地址映射（直接映射）、4.25A Cache——地址映射（全相联映射）、4.25B Cache——地址映射（组相联映射））；读写流程（4.22 Cache——读写流程）；习题课（4.27 Cache——替换算法（习题课一）、4.28 Cache——替换算法（习题课二））